University of Portsmouth

School of Creative Technologies

Final Year Project

Evaluating the approachability of scientific papers towards creating a simulated particle physics scenario in C++ without prior background in the field

by

Tayler Parsons
UP2212801

Supervisor: Jahangir Uddin

April 19, 2026

Project Type: Combined

I consent to my report in this attributed format (not anonymized), subject to final approval by the Board of Examiners, being made available in the Library Dissertation Repository and/or the school digital repositories for a maximum of 10 years.

✓Yes No

This report investigates the accessibility of scientific literature from a software engineering perspective, using the creation of a sub-atomic particle physics simulation as a case study. Using physics research, a simulation will be made that aims to realistically reproduce the behaviour of the fundamental particles, including Protons, Neutrons, Electrons. Furthermore, the simulation will attempt to represent the four fundamental interactions: the strong, weak, electromagnetic, and gravitational forces.

Through the implementation of these concepts without prior academic or professional expertise, this project aims to evaluate the translatability and level of understanding needed to take the mathematics and concepts discussed into code.

Contents

1 Introduction **2**

1.1 Project Aim 2

1.2 Subjectivity Disclaimer 2

2 Project Management & Time Keeping **3**

3 Literature Review **4**

3.1 Introduction 4

3.2 Review of existing literature 4

3.2.1 Foundational Physics 4

3.2.2 Computational Physics 5

3.2.3 Developer Experience & Accessibility 5

3.2.4 Selected Studies on Particle Physics Simulation 5

3.2.5 Summary 9

3.2.6 Selected Particle Physics Simulations 9

4 Accessibility Metric System **12**

4.1 Scoring Stystem 12

4.1.1 Evaluation Process 12

5 Methodology **14**

5.1 Simulation Architecture 14

5.1.1 Code Repository 14

5.2 Approachability Audit 14

5.2.1 Data Collection Process 14

5.2.2 Evaluation 15

5.2.3 Problem with Units & Scales 15

5.3 Fundamental Forces 16

5.3.1 Gravitational 16

References **20**

Chapter 1

Introduction

Software engineering has always been closely tied to physics and mathematics to the point where the former has become strongly associated with the latter. In some fields, such as game development which has a very large independant / solo community, this may be considered to be a barrier to entry for some. People may be dissuaded from learning how to program due to how heavy the mathematics might be, leading to the field missing out on great games made from the passion of talented people.

Resources such as research papers which are not explicitly designed with beginners-to-the-field, or game developers mystify the mathematics due to heavily verbose notation and content.

1.1 Project Aim

This project aims to evaluate and review how approachable research papers are for people who do not have a background in mathematics and physics, in creating a project.

The aim of this project is to create a plausible simulation of particle physics in C++, a project that will be heavily based in foundational physics.

1.2 Subjectivity Disclaimer

The evaluation of the accessibility and approachability of scientific papers and by extension the developer experience detailed in this report is inherently subjective, shaped by my own specific background in both particle physics (or lack thereof), and experience in programming, and interpretation of the resources used. As such, the challenges and successes documentated will vary for other developers with different backgrounds and levels of expertise in the two fields.

Chapter 2

Project Management & Time Keeping

Due to extenuating circumstances, development on this artefact and paper did experience a significant delay. I had initally spent some significant time at the beginning of the project creating the boilerplate code for the simulation, which was partially adapted from Marmalade Engine (Parsons & Bester, 2026), a modular Game Engine co-developed by myself.

Marmalade Engine was used as base due to my familiarity with the core structure, and to eliminate much of the time and effort it would take if I was to do it from scratch. Furthermore, it should be noted that the code used was primarily the rendering backend, whereas nothing relating to the development of the simulation was used.

Due to weak prioritisation of my other university commitments, and the shortcomings of development early on in the supposed project timeline, the primary development window took place across the entire month of April, 2026.

Chapter 3

Literature Review

3.1 Introduction

The purpose of a literature review is to identify existing areas of research relating to the given topic, alongside the any areas that are lacking in research literature. It also serves to provide an understanding of the topics that are intended to be discussed and expanded upon during the project.

3.2 Review of existing literature

Scientific literature regarding any subset of physics is traditionally written for those with a strong background of physics, mathematics, or research. Because of this, people outside of such disciplines, such as those in computer science including the game development fields, may be discouraged to use these resources due to the formatting, terminology or mathematic symbology that may unfamiliar to them. The result is developers who miss out on learning new techniques, algorithms, or optimisation that may benefit their software or game. This literature review, and the following report evaluates foundational physics papers and their accessibility to those not discipined in physics or mathematics through the creation of a particle physics simulation in C++.

This review aims to source research within particle physics, and computer science.

3.2.1 Foundational Physics

This first category of literature consists of foundational physics papers, such as *A Standard Model Workbook* (Moore, 2024, p. .) While reports such as these serve to explain and give context to the bedrock of particle physics, they do so whilst presenting a developer experience barrier due to how its content is often dense mathematics.

From a software engineering perspective, the primary concern would be the use of abstract notation which, while logically might be familiar, are syntaxtically unorthodox. An example of this is $\sum$ which represents iterative accumulation, comparable that to a programmatical *for*

loop. Because of this notation, software and game developers would first need to translate the content from the paper into pseudo code.

3.2.2 Computational Physics

Computational literature such as *Computer Simulation in Physics and Engineering* (Steinhauser, 2012, a)re often based on the implementation of the theory established by foundational literature. Because of this, it is less mathematically dense, and more focal on the practical implementation of the functions and concepts introduced.

However, with this particular example, the argument could be made that it has the inverse problem than the foundational paper mentioned above, focusing heavily on the basics of computer science, such as explaining binary and arithmetic-in-code, rather than the actual implementation of the mathematics.

3.2.3 Developer Experience & Accessibility

The disparity between these two types of academic literature creates a gap between the the Developer Experience (**DX**), which can result in longer development time as considerable time may be required to 'reverse-engineer' the mathematics.

Furthermore, it should be noted that aims of the reseacher and developer may differ.

While the researcher may be showing the metholodgy used to achieve a specified result. The developer often times would also also aim for efficiency and scalablity in the algorithms used; something that the researcher would not have considered.

3.2.4 Selected Studies on Particle Physics Simulation

The following subsections present a comparison of a series of papers that relate to particle physics simulation, and, where possible, address how accessible their content are for non-specialists.

Computer simulation in data analysis: A case study from particle physics

In the paper "Computer Simulation in Data Analysis" (Falkenburg, 2024, a) framework for processing data from large particle colliders is presented. The paper includes key terms and notation such as:

1. Likelihood function: $\mathcal{L}(\theta) = \prod_u P(x_i|\theta)$
2. Bayesian prior: $\pi(\theta)$
3. Markov Chain Monte Carlo: $\alpha = \min\left(1, \frac{\pi(\theta')}{\pi(\theta)}\right)$

Accessibility

- *Lexical*: 'Likehood' and 'Prior' terms are only defined in footnotes, which may go unnoticed

- *Notational*: Product symbol and superscript notation assume that the reader is familiar with certain mathematical notation and probablity theory.

This paper arguably demonstrates that well-structured simulation papers can still be inacessible to developers.

Differentiable simulations for particle tracking in accelerators: Analysis, benchmarking, and optimization

In the paper "Differentiable Simulations for Particle Tracking in Accelerators" (Huhn & Velotti, 2025) a framework is introduced to allow for gradient-based optimisation of particle trajectories. The paper includes key terms and notation such as:

1. Hamiltonian $H(\mathbf{z})$, a function of phase-space vector $\mathbf{z}$.
2. Symplectic integrator
3. Gradient of the loss function $\nabla_\theta L(\theta)$

Accessibility

- *Notational*: Heavy use of matrix algebra and Poisson brackets
- *Conceptual*: Phase-space dynamics may be unfamiliar to computer science and game programmers.

Particle simulation study on the transport of laser-accelerated proton beams in high-density plasmas

In the paper "Particle Simulation Study on the Transport of Laser-Accelerated Proton Beams in High-Density Plasmas" (Xu et al., 2024), a model is presented showing proton acceleration in dense plasmas using particle-in-cell (PIC) techniques. The paper includes key terms and notation such as:

1. Electric field $E = -\nabla\Phi$
2. Lorenz force $\mathbf{F} = q(\mathbf{E} + \mathbf{v} \times \mathbf{B})$
3. Normalised units (e.g., a_0, ω_{pe})

Accessibility

- *Notational*: Cross product notation and normalised units
- *Conceptual*: Plasma frequency ω_{pe} and normalisation may be unfamiliar in a physics context

Simulation Study on Closed-orbit Correction for Storage Ring of Wuhan Advanced Light Source

The paper "Simulation Study on Closed-orbit Correction for Storage Ring of Wuhan Advanced Light Source" (Qiao et al., 2026) presents an algorithm to correct closed-orbit correction for the storage ring of the Wuhan Advanced Light Source, focusing on beam stability. The paper includes key terms and notation such as:

1. Closed-orbit deviation $\Delta x(s)$
2. Orbit response matrix R_{ij}
3. Singular value decomposition (SVD)

Accessibility

- *Notational*: Use of matrix calculus and SVD may be unfamiliar to some game developers
- *Conceptual*: The idea of a 'closed orbit' and its physical significance may not be initially known

Quantum simulation of 2D topological physics in a 1D array of optical cavities

The paper "Quantum Simulation of 2D Topological Physics in a 1D Array of Optical Cavities" (Luo et al., 2015) demonstrates a quantum simulation of 2D topological phases using a 1-D array of coupled optical cavities. The paper includes key terms and notation such as:

1. Tight-binding Hamiltonian $H = -J \sum_{\langle i,j \rangle} (a_i^\dagger a_j + \text{h.c})$
2. Berry curvature $\Omega(k)$
3. Chern number $C = \frac{1}{2\pi} \int_{\text{BZ}} \Omega(k)\, dk$.
4. Pauli matrices $\sigma_x, \sigma_y, \sigma_z$

Accessibility

- *Notational*: Heavy used of second-quantised operators, integrals, and complex matrix notation.
- *Conceptual*: Topics such as the Chern number and Berry curvature are not commonly studied topics in computer science or game development

Performance Analysis of C and Python Parallel Implementations on a Multicore System Using Particle Simulation

The paper "Performance Analysis of C and Python Parallel Implementations on a Multicore System Using Particle Simulation" (Asaduzzaman et al., 2024) compares C++ and Python implementation of a particle simulation on a multicore system, and focuses on optimisation, memory usage, and scalability. The paper includes key terms and notation such as:

1. OpenMP pragmas (e.g '#pragma omp paralell for')
2. GIL (Global Interpreter lock)
3. Speed-up factor $S = T_{\text{serial}}/T_{\text{parallel}}$.
4. Amdahl's Law: $S_{\max} = 1/(1 - P + P/N)$.

Accessibility

- *Lexical*: Terms such as 'GIL' and 'OpenML' are specifgic to HPC and may be unknown.
- *Notational*: Speed-up formulae involve fractions and variables that may be unfamiliar.

Simulation of matter–antimatter creation on quantum platforms

The paper "Simulation of Matter–Antimatter Creation on Quantum Platforms" (Burrello, 2025) proposes a quantum algorithm to simulate matter-antimatter pair creation The paper includes key terms and notation such as:

1. Qubit state $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$.
2. Controlled-NOT gate
3. Trotterisation
4. Quantum volume V_Q

Accessibility

- *Notational*: Dirac notation, gate symbols, and Trotterisation forumula may be unfamiliar to some game developers. However, the those with a computer science background are more likely to understand gate symbols
- *Conceptual*: The physical process of pair creation and its integration onto a quantum circuit may require background knowledge in high energy physics.

3.2.5 Summary

The selected studies indicate the particle-physics simulation papers usually contain notation and terminonaly that can be considered verbose or inaccessible to devlopers with a background in physics, or mathematics.

None of these papersd provide an audit of these notational barriers, nor do they link such barriers to an actual effort at implementation.

And as such, this review has provided an insight to the current landscape of particle physics simulation literature, with identified key accessibility issues, and a potentionally justified need for an audit, or key of notation and terminology used.

The subsequent chapters will present the methodology used to create a particle physics simulation, using relevant literature to provide a more practical implementation of the mathematics written about in particle physics simulation literature.

3.2.6 Selected Particle Physics Simulations

The following table summaries five Github projects that implement particle or sub-atomic particle simulations. All of which are open-source and written in a language that are approachable to software engineers such as C++ or Python.

Table 3.1: Community Particle Simulation Reference Table

Repository	Language	Visualisation	Accessibility
Monte-Carlo Particle Simulator	C	None (CLI)	Very lightweight, however documentation requires compilation
Charged-Subatomic-Particle-Interaction-Simulator	Python	Video	Physics equations buried in a single file, documentation non-existent
Quantum Collider Sandbox	Python	3D	Very comprehensive documentation
APRIL - A Particle Runtime Interaction Library			
OpenWave			

Monte-Carlo Particle Simulator (gyaikhom)

The simulation "Monte-Carlo-Particle-Simulator" (gyaikhom, 2014) implements a Monte-Carlo particle physics simulation model, specifically designed for high-energy physics.

Accessibility

- This project must be compiled, which may restrict some developers who are not familiar with compiling applications from source, from using it.
- The project is quite dated, with the last commit being from 12 years ago. This can make it challenging to compile due to the outdated CMake version it requires.
- CUDA is required to compile the project, which can prove to be a another barrier for a developer to use if they are unable to, or does not know, how to acquire it.

Due to this simulation outputting its result in the terminal, it may be hard to visualise the data that is being passed passed into it. Given this, and its age, it can be confidently stated that this simulation may not be approachable for a developer to use.

Charged-Subatomic-Particle-Interaction-Simulator (RaresC21)

The simulation "Charged-Subatomic-Particle-Interaction-Simulator" (Cristian, 2017) implements a simple model for displaying charged particle trajectories in a uniform magnetic field. It models protons and electrons but does **not** include neutrons or whole atoms/isotopes. This simulation takes an interesting approach for its visualisation, opting for a *.mp4* video rendered using "FFmpeg" (Bellard, 2000) to display the results.

Accessibility

- This project requires Python to run.
- Given the project being written in Python, it is quite easy to modify the code.
- Documentation is non-existent. Requiring the user to reverse-engineer the code to understand how it works and modify it to thier preference.

Considering that this project only requires Python, a language found on most computers, this simulation is arguably quite accessible. However, the lack of documentation and explanatory code comments makes this project harder to modify to the users needs. Furthermore, the burdersome act of having to run the python script, then opening a *.mp4* file could prove unappealing to developers who would prefer a program where they can perform quick, iterative changes.

quantum-collider-sandbox (ml3m)

The simulation "Quantum-Collider-Sandbox" (mlem, 2026) presents a 3D Sandbox scene which allows the user to view and manipulate various settings

Accessibility

- This project requires Python to run.

- Given the project being written in Python, it is quite easy to modify the code.
- Documentation is very comprehensive. A web page is provided which includes an installation guide and a usage guide.
- The simulation includes a user interface.
- Some editable fields have vague names.

This simulation is very accessible. Due to this project only requiring Python to run, it is very easy to set up and run. Alongside this, the accessible comprehensive documentation helps to provide the user a very powerful sandbox.

Some of the editable fields in the interface suffer from vague names such as *'B_x'*, *'B_y'*, and *'E_x'*. These are explained as the magnetic and electromagnetic fields, this lack of immediate explanatory variable naming is very sparse. It is not apparent if the user can 'spawn' in particles by themselves, which may hinder the accessibility when creating any unique situations that the user may desire.

APRIL A Particle Runtime Interaction Library (JKKDY)

This framework "April" (Deller-Yee, 2026), aims to provide the resources to create high-performance particle-based simulations. Whilst not strictly being a 'simulation', it provides the tools necessary to create them.

Accessibility

- This project is newer compared to the aforementioned projects. Last being updated in March 2026.
- Documentation is provided, including basic example projects and performance statistics.
- Being a framework requires more setup than other simulations, which is detrimental for the accessibility of this project.
- Since this is a C++ framework, some developers may struggle to use it given the complexity of the language.

Chapter 4

Accessibility Metric System

To provide a basis of the subjective development experience, each source will be subjected to an analysis which tracks the frequency and severity of instances that the source is lacking in clarity and requires switching to other resources to understand, or add to, the context written.

4.1 Scoring System

Each source will accumulate points depending on how far and how long I need to refer to other sources to understand the original.

Table 4.1: Complexity Severity Levels

Level	**Value**	**Description**	**Examples**
Minor	1	Lexical issue. A word, phrase, or term that may not be immediately understood, requiring brief additional research.	'Isospin', 'Alpha Decay'
Moderate	2	Notation barrier. Mathematical notation that is not explained, requiring research into determining its meaning.	$\sum$
Major	4	Unexplained introduced concepts. Concepts that are mentioned but not elaborated, which may require extensive further research.	—
Critical	8	Incomprehensive topic. A topic is difficult to research or understand in the time given; e.g., a proof with no numerical constants.	Complex proofs

4.1.1 Evaluation Process

As the particle simulation is developed, each time the workflow is interrupted with needing to find context through additional resources, points will be accumulated and logged for each major resource, and a final **Approachability Score** will be created using the following equation:

$$A = \sum (P_{\text{minor}} + P_{\text{moderate}} + P_{\text{major}} + P_{\text{critical}}) \tag{4.1}$$

A lower Approachability Score indicates a more developer-friendly resource. Whereas a higher score suggests a more difficult resource for developers to use.

Chapter 5

Methodology

The methodology of this project will be double-faceted, focusing on the development of the simulation itself, and the concurrent audit of sources used to aide that development.

5.1 Simulation Architecture

The simulation is developed using C++ due to its high performance and stability, and memory management which will be important for handling large particle counts and their calculations efficiently.

- **Object-orientated design:** Each particle will be considered it's own object. Handling it's own position, radius, and systems such as rendering.
- **Environment:** Development is performed in a Linux environment using the GCC compiler.

The simulation also uses Dear ImGui (commonly referred to as *ImGui, imgui, or IMGUI*) to render a Graphical User Interface (GUI), this will provide the user statistics such as the number of particles, performance stats, alongside providing the user the ability to add or remove particles. (omar, 2026)

5.1.1 Code Repository

The code for the simulation can be found for free, on my public GitHub repository (taylerparsons, 2026)

5.2 Approachability Audit

5.2.1 Data Collection Process

As additions are made to the simulation (e.g the gravitational force), the primary source that was used for that feature will be logged, and its approachability will be scored and logged based on the criteria shown in table 4.1.

5.2.2 Evaluation

At the end of the development of the simulation, the **Approachability Score *AS*** will be calculated and cross-referenced to the estimated time it had taken to implement the feature the source was used for.

5.2.3 Problem with Units & Scales

The first issue encountered was the ubiquitous application of the International Systemm of Units **SI** (National Institute of Standards and Technology, 2010), which standardises the accepted use, measurement, and notation of quantaties, including mass, weight, speed, and distance.

The use of SI Units by scientific literature creates a problem when translating equations to code. This is because many values, such as the virtually infinitesimal mass of a proton: 1.67×10^{-27} kg become difficult to represent by a computer, often leaving to floating-point precision errors.

To avoid the imprecision of computational small numbers, we must normalise the SI Units into a more manageable scale. These units we will call **Simulation Units** or su. By selecting a base reference, such as the mass of a proton, we can define a new coordinate system where all other values are represented relative to that base reference.

For the simulation, the mass of a Proton (m_p) was selected as the fundemental unit of measurement ($1.000su$). All other masses are converted by dividing their SI value by m_p.

Table 5.1: SI to SU Conversion Examples

Name	SI Unit (kg)	Equation	Converted Simulation Unit (su)
Proton	1.67×10^{-27}	$(1.67 \times 10^{-27}) \div (1.67 \times 10^{-27})$	1.000
Neutron	1.67×10^{-27}	$(1.67 \times 10^{-27}) \div (1.67 \times 10^{-27})$	1.000
Electron	9.11×10^{-31}	$(9.11 \times 10^{-31}) \div (1.67 \times 10^{-27})$	0.0005

5.3 Fundamental Forces

Now that the problem with units and scales has been addressed, the first major feature could be implemented, the Fundemental Forces. For this, I heavily relied on *A Modern Introduction to Particle Physics* (Fayyazuddin & Riazuddin, 2011, f)or the base of the mathematics and general understanding.

At this point the project boilerplate has been written, such as the imgui interface, and the ability to place either a *red* proton *(p)*, *blue* neutron *(n)*, *yellow* electron *(e^-)*, or a *purple* photon *(ph)*, into the sandbox scene.

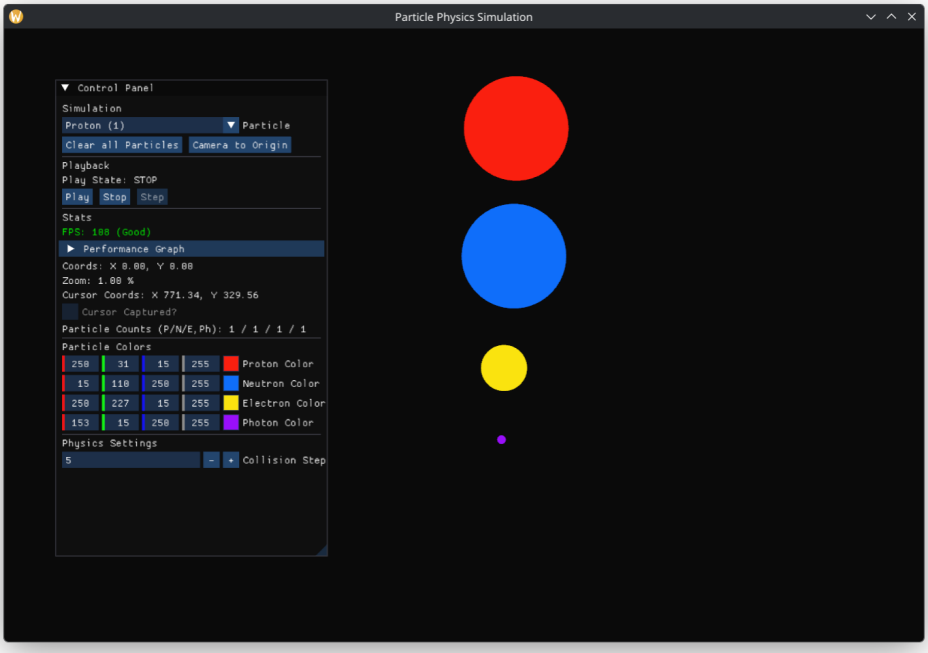

Figure 5.1: App window showing imgui interface and the particles

5.3.1 Gravitational

The gravitational force, stated by Fayyazuddin and Riazuddin (2011) (p. 2), is a 'force of attraction between two particles and is proportional to their gravitational charges, namely their masses". This definition was very easy to understand, stating that the force is the how attracted two particles are given their mass.

However, when the implementation of the force began, I quickly ran into the first **Lexical** issue. The paper noted that the strength of the gravitational force was:

$$G_n = 1.3 \times 10^{-54} \, (GeV/m)^{-1} \tag{5.1}$$

Fayyazuddin and Riazuddin, 2011, p. 2

The first half of this equation made sense: $G_n = 1.3 \times 10^{-54}$, but what was $(GeV/m)^{-1}$?

The presence of the yet-unknown unit GeV cause quite a bit of misunderstanding. However, quick research proved that this unit measure one **Electron Gigavolt**.

An GeV is equivelent to a billion electron volts. A unit used to indicate the gain, or loss of kinetic energy an electron experiences when traversing through an electric potentional difference.

Given how straight-forward and quick it was to research this, I assigned this an accessibility score of 1.

List of Figures

5.1 App window showing imgui interface and the particles 16

List of Tables

3.1 Community Particle Simulation Reference Table 9

4.1 Complexity Severity Levels 12

5.1 *SI* to *SU* Conversion Examples 15

References

Asaduzzaman, A., Telikepalli, V. S. P. T., & Uddin, M. R. (2024). Performance Analysis of C and Python Parallel Implementations on a Multicore System Using Particle Simulation. *2024 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA), Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA), 2024 International Conference on*, 1–7. https://doi.org/10.1109/ACDSA59508.2024.10467885

Bellard, F. (2000, December 20). *FFmpeg*. Retrieved April 19, 2026, from https://ffmpeg.org/

Burrello, M. (2025). Simulation of matter–antimatter creation on quantum platforms. *Nature*, *642*(8067), 308–309. https://doi.org/10.1038/d41586-025-01581-3

Cristian, R. (2017, April 24). *Charged-Subatomic-Particle-Interaction-Simulator*. Retrieved April 19, 2026, from https://github.com/RaresC21/Charged-Subatomic-Particle-Interaction-Simulator

Deller-Yee, J. (2026, March 25). *April*. Retrieved April 19, 2026, from https://github.com/JKKDY/april

Falkenburg, B. (2024). Computer simulation in data analysis: A case study from particle physics. *Studies in history and philosophy of science*, *105*, 99–108. https://doi.org/10.1016/j.shpsa.2024.03.005

Fayyazuddin & Riazuddin. (2011). *A modern introduction to particle physics* (3rd ed.). World Scientific Publishing Company.

gyaikhom. (2014, May 26). *Monte-carlo-particle-simulator*. GitHub. Retrieved April 19, 2026, from https://github.com/gyaikhom/monte-carlo-particle-simulator

Huhn, F., & Velotti, F. M. (2025). Differentiable simulations for particle tracking in accelerators: Analysis, benchmarking, and optimization. *Physical Review Accelerators and Beams*, *28*(11), 114603–114603. https://doi.org/10.1103/cbds-lst1

Luo, X.-W., Zhou, X., Li, C.-F., Xu, J.-S., Guo, G.-C., & Zhou, Z.-W. (2015). Quantum simulation of 2D topological physics in a 1D array of optical cavities. *Nature Communications*, *6*(1), 7704. https://doi.org/10.1038/ncomms8704

mlem. (2026, April 9). *Quantum-collider-sandbox*. Retrieved April 19, 2026, from https://github.com/ml3m/quantum-collider-sandbox

Moore, T. A. (A. (2024). *A standard model workbook*. University Science Books.

National Institute of Standards and Technology. (2010, April 12). *Si units* [Last modified 2025-07-22]. National Institute of Standards and Technology. Retrieved April 8, 2026, from https://www.nist.gov/pml/owm/metric-si/si-units

omar. (2026, April 3). *Ocornut/imgui*. Retrieved April 3, 2026, from https://github.com/ocornut/imgui

Parsons, T., & Bester, R. (2026, March 11). *Marmalade-Engine/marmalade*. Retrieved April 8, 2026, from https://github.com/Marmalade-Engine/marmalade

Qiao, X., Yu, X., Huang, L., He, T., Liu, G., Zou, Y., Dai, Z., Li, J., Nie, Y., & He, J. (2026). Simulation study on closed-orbit correction for storage ring of Wuhan Advanced Light Source. *Yuanzineng kexue jishu*, *60*(2), 489–496. https://doi.org/10.7538/yzk.2025.youxian.0340

Steinhauser, M. O. (2012). *Computer Simulation in Physics and Engineering*. De Gruyter, Inc. Retrieved March 4, 2026, from http://ebookcentral.proquest.com/lib/portsmouth-ebooks/detail.action?docID=893213

taylerparsons. (2026, April 17). *UpsilonDiesBackwards/ParticlePhysicsSim*. Retrieved April 17, 2026, from https://github.com/UpsilonDiesBackwards/ParticlePhysicsSim

Xu, W. W., Hu, Z. H., Zhao, Y. T., & Wang, Y. N. (2024). Particle simulation study on the transport of laser-accelerated proton beams in high-density plasmas. *2024 IEEE International Conference on Plasma Science (ICOPS), Plasma Science (ICOPS), 2024 IEEE International Conference on*, 1–1. https://doi.org/10.1109/ICOPS58192.2024.10626678